

Practical 2

Prop Drilling

Aim:

To refactor a Vue 3 application into a nested component structure and demonstrate data communication using props (passing data down)

Theory:

1. Component Nesting

In modern web development, user interfaces are divided into reusable, self-contained building blocks called **components**. These components can be nested inside one another to form a tree structure (Hierarchy):

- **Parent Component:** App.vue
 - **Child Component:** Dashboard.vue
 - **Grandchild Components:** StudentList.vue & AddStudentForm.vue
 - **Great-Grandchild Component:** StudentCard.vue

2. Props (Passing Data Down)

In Vue, data is passed from a parent component down to a child component using **props**. Props are read-only attributes that allow the child to access data belonging to the parent.

Syntax:

```
Parent: <ChildComponent :customProp="parentState" />
Child: defineProps(['customProp'])
```

In this practical, props are passed across multiple hops: App.vue (master students array)
=> Dashboard.vue => StudentList.vue => StudentCard.vue (individual student).

3. Emits (Passing Actions Up)

Since props are read-only, a child component cannot modify the parent's data directly. Instead, when an action occurs (like clicking a delete button or submitting a form), the child **emits** a custom event upwards. The parent listens to this event and modifies its local state.

Syntax:

```
Child: emit('customEvent', payload)
Parent: <ChildComponent @customEvent="handleAction" />
```

In this practical, custom events bubble upwards sequentially without skipping levels: StudentCard.vue (delete click) => StudentList.vue => Dashboard.vue => App.vue (filters the array).

Code Snippets:

1. Import Statements

```
import { ref } from "vue";
import users from "../users.json";
import Dashboard from "../components/Dashboard.vue";
```

2. Master Student State (App.vue)

```
const students = ref([
  { id: 1, name: "Ethan Rodrigues", age: 20, grade: "A" },
  { id: 2, name: "Peter Parker", age: 21, grade: "B" },
  { id: 3, name: "Ben Reilly", age: 22, grade: "A" }
])
```

3. Prop Hops (passing data down to components)

- **Hop 1: App to Dashboard**

```
<Dashboard
  :role="currentUser.role"
  :students="students"
  :username="currentUser.username"
/>
```

- **Hop 2: Dashboard to StudentList**

```
<StudentList
  :students="students"
  :role="role"
/>
```

- **Hop 3: StudentList to StudentCard (v-for iteration)**

```
<StudentCard
  v-for="student in students"
  :key="student.id"
  :student="student"
  :role="role"
/>
```

4. Emit Events (Passing Actions Upwards)

- **Step 1: StudentCard Emits 'delete' on button click**

```
<button v-if="role === 'Admin'" @click="$emit('delete', student.id)">
  Delete
</button>
```

- **Step 2: StudentList captures and bubbles event up to Dashboard**

```
<StudentCard @delete="$emit('remove-student', $event)" />
```

- **Step 3: Dashboard bubbles event up to App.vue**

```
<StudentList @remove-student="$emit('remove-student', $event)" />
```

- **Step 4: App.vue captures event and filters array state**

```
<Dashboard @remove-student="handleRemoveStudent" />
javascript
function handleRemoveStudent(id) {
  students.value = students.value.filter(s => s.id !== id);
}
```

5. Form Model Bindings and Emitter (AddStudentForm.vue)

```
<form @submit.prevent="submitForm">
  <input type="text" v-model="name" placeholder="Enter student's full
name" required />
  <input type="number" v-model.number="age" min="1" required />
  <select v-model="grade" required>...</select>
  <button type="submit">Add Student</button>
</form>
function submitForm() {
  emit('add-student', {
    id: Date.now(),
    name: name.value.trim(),
    age: age.value,
    grade: grade.value
  });
}
```

Output:

Welcome, admin!

Admin Dashboard

Logout

Add New Student

Name:

Enter student's full name

Age:

Enter age

Grade:

Select Grade

Add Student

Registered Students (3)

Average Age: 21.0 years

Ethan Rodrigues

Age: 20
Grade: A

Delete

Peter Parker

Age: 21
Grade: B

Delete

Ben Reilly

Age: 22
Grade: A

Delete

1) Admin Dashboard

Welcome, admin!

Admin Dashboard

Logout

Add New Student

Name:

Enter student's full name

Age:

Enter age

Grade:

Select Grade

Add Student

Registered Students (4)

Average Age: 21.3 years

Peter Parker

Age: 21
Grade: B

Delete

Ben Reilly

Age: 22
Grade: A

Delete

Jean Grey

Age: 22
Grade: C

Delete

2) Adding a New Student

Welcome, user!

User Dashboard

Logout

Registered Students (4)

Average Age: 21.3 years

Ethan Rodrigues Age: 20 Grade: A
Peter Parker Age: 21 Grade: B
Ben Reilly Age: 22 Grade: A
Jean Grey Age: 22 Grade: C

3) User Dashboard